

Experiment 2

System Architecture, Database Schema, Process Flow & Technology Stack

Name: Abhijay Das | Roll No: 49

Course: E-Commerce & E-Business | VESIT, Mumbai

1. System Architecture Design

1.1 Architecture Overview

TechNest follows a Three-Tier Architecture Model that cleanly separates the system into three independent layers. This design promotes modularity, scalability, and easier maintenance.

Presentation Layer	• React.js with Tailwind CSS for dynamic, responsive UI • Manages user interactions: browse, search, filter, cart, checkout • Communicates with backend exclusively via REST APIs (JSON over HTTPS) • State management using React Context API / Redux
Application Layer	• Java 17 + Spring Boot 3.x as the core backend framework • Spring Web MVC handles REST API routing and controllers • Spring Security + JWT manages authentication and authorization • Spring Data JPA (Hibernate) as the ORM layer • Business logic: order processing, inventory updates, payment handling • Razorpay Java SDK integrated for payment gateway
Data Layer	• PostgreSQL (relational database) for structured data storage • Tables: users, products, categories, orders, order_items, reviews • Hibernate manages entity mapping and schema generation • Connection pooling via HikariCP for optimal performance • Supabase (cloud PostgreSQL) for production deployment

1.2 Architecture Flow

The client (React frontend) sends HTTP requests to the Spring Boot REST API. The API layer authenticates the request via JWT, applies business logic, and communicates with the PostgreSQL database using JPA repositories. Responses are returned as JSON payloads to the frontend.

Frontend (React + Tailwind) → REST API (Spring Boot) → JPA / Hibernate → PostgreSQL (Supabase)

2. Technology Stack Selection

Layer	Technology	Version	Purpose
Frontend	React.js	18.x	Component-based dynamic UI
Styling	Tailwind CSS	3.x	Utility-first responsive design
Backend	Java	17 (LTS)	Core backend programming language
Framework	Spring Boot	3.x	REST API and app configuration
API Layer	Spring Web MVC	3.x	HTTP routing and controllers
Security	Spring Security + JWT	3.x	Auth, role-based access control
ORM	Hibernate / Spring Data JPA	3.x	Database entity mapping
Database	PostgreSQL	15.x	Relational data storage
DB Hosting	Supabase	Cloud	Managed PostgreSQL in cloud
Payment	Razorpay Java SDK	Latest	Sandbox payment integration
Build Tool	Maven	3.9	Dependency & build management
Version Control	Git & GitHub	-	Source code management
Frontend Deploy	Vercel	Cloud	Static frontend hosting
Backend Deploy	Railway	Cloud	Spring Boot app hosting

2.1 Reasons for Technology Selection

- Java + Spring Boot is an industry-standard, enterprise-grade backend widely used in production e-commerce systems.
- Spring Security provides robust, battle-tested authentication and authorization out of the box.
- PostgreSQL is a powerful relational database, ideal for structured e-commerce data with complex relationships.
- Hibernate/JPA simplifies database operations using Java objects, eliminating raw SQL for standard operations.
- React.js enables a fast, interactive frontend with reusable components and efficient DOM rendering.
- Railway and Vercel offer simple, low-cost deployment pipelines with GitHub integration.

3. Database Schema Design

TechNest uses PostgreSQL with a fully normalized relational schema. The following tables are defined:

3.1 users Table

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique user identifier

username	VARCHAR(100)	NOT NULL	Customer's display name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt-hashed password
role	VARCHAR(20)	DEFAULT 'USER'	USER or ADMIN role
phone	VARCHAR(15)	NULLABLE	Contact number
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time

3.2 categories Table

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique category ID
name	VARCHAR(100)	NOT NULL, UNIQUE	Category name (e.g. Laptops)
description	TEXT	NULLABLE	Category description

3.3 products Table

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique product ID
name	VARCHAR(255)	NOT NULL	Product name
brand	VARCHAR(100)	NOT NULL	Manufacturer/brand name
category_id	BIGINT	FK → categories.id	Product category
price	DECIMAL(10,2)	NOT NULL	Selling price
description	TEXT	NULLABLE	Detailed product description
image_url	VARCHAR(500)	NULLABLE	Product image link
stock	INT	DEFAULT 0	Available stock quantity
rating	DECIMAL(3,2)	DEFAULT 0.0	Average customer rating
created_at	TIMESTAMP	DEFAULT NOW()	Product listing date

3.4 orders Table

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique order ID
user_id	BIGINT	FK → users.id	Customer who placed order
total_amount	DECIMAL(10,2)	NOT NULL	Final order total
status	VARCHAR(30)	DEFAULT 'PENDING'	PENDING / SHIPPED / DELIVERED

payment_id	VARCHAR(100)	NULLABLE	Razorpay payment reference
payment_status	VARCHAR(20)	DEFAULT 'UNPAID'	UNPAID / PAID / FAILED
created_at	TIMESTAMP	DEFAULT NOW()	Order placement timestamp

3.5 order_items Table

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique order item ID
order_id	BIGINT	FK → orders.id	Parent order reference
product_id	BIGINT	FK → products.id	Product purchased
quantity	INT	NOT NULL	Number of units ordered
unit_price	DECIMAL(10,2)	NOT NULL	Price at time of purchase

3.6 reviews Table

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique review ID
product_id	BIGINT	FK → products.id	Product being reviewed
user_id	BIGINT	FK → users.id	Reviewer's user account
rating	INT	CHECK (1–5)	Star rating (1 to 5)
comment	TEXT	NULLABLE	Written review content
created_at	TIMESTAMP	DEFAULT NOW()	Review submission date

3.7 Entity Relationships

- One User can place many Orders (1:N)
- One Order can contain many Order Items (1:N)
- One Product belongs to one Category (N:1)
- One Product can appear in many Order Items (1:N)
- One User can write many Reviews; one Product can have many Reviews (1:N each)

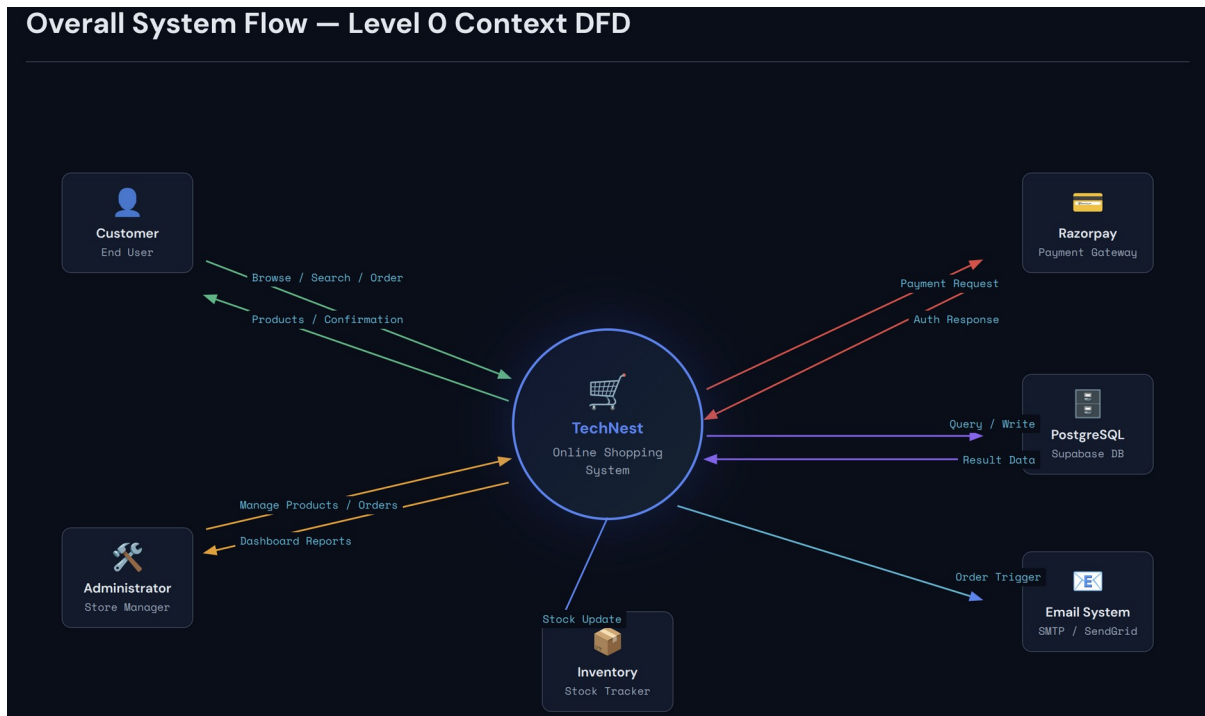
4. Process Flow Diagrams

4.1 Overall System Flow (Level 0 DFD)

At the highest level, TechNest interacts with the following external actors:

Actor	Interaction with System
-------	-------------------------

Customer	Browses catalog, manages cart, places orders, tracks delivery
Administrator	Manages products, categories, inventory, and order statuses
Razorpay Gateway	Receives payment requests, returns authorization response
Email System	Receives order confirmation trigger, sends email to customer
Supabase (DB)	Persists and retrieves all application data



4.2 Detailed User Flow (Level 1 DFD)

User Registration & Login

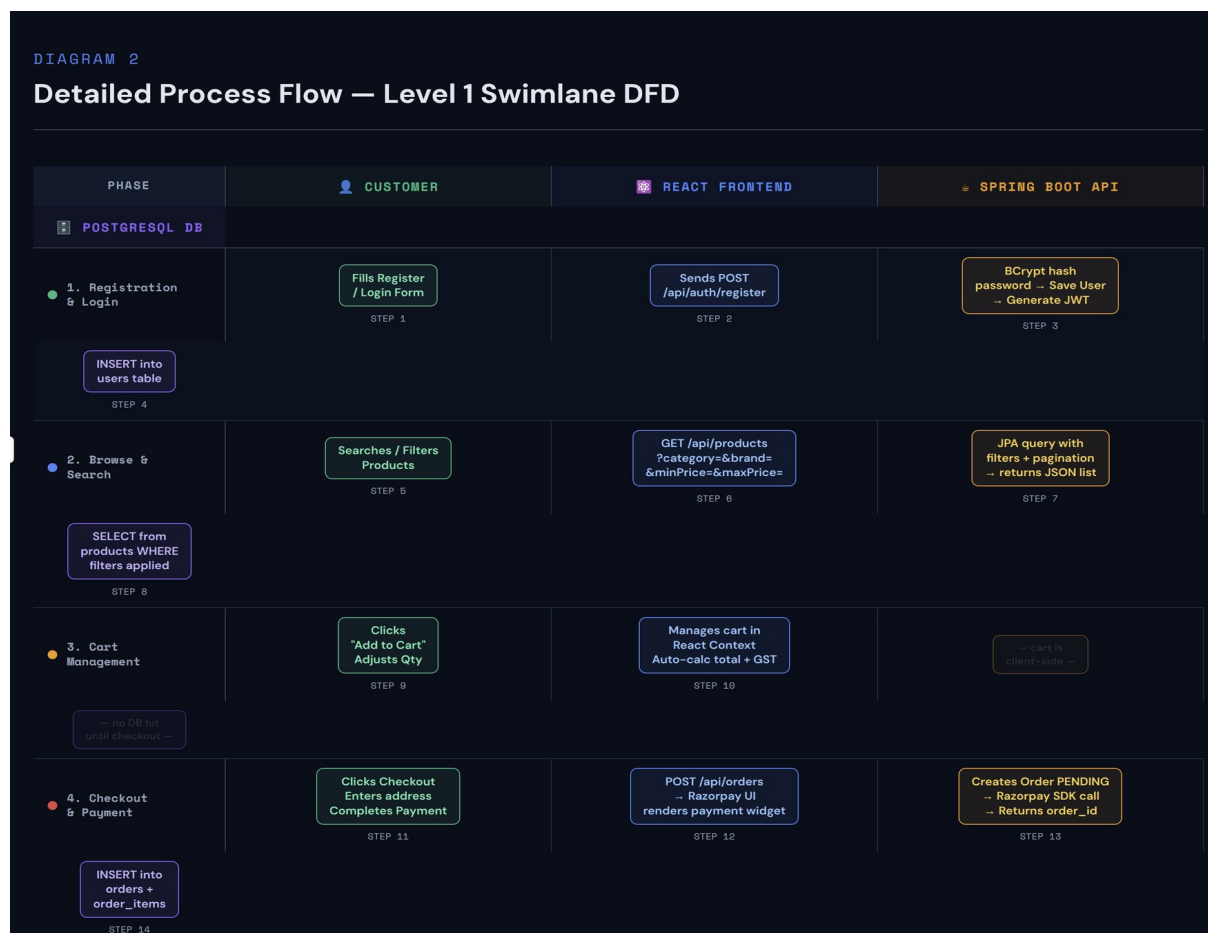
1. User submits registration form with name, email, and password.
2. Spring Boot receives the request at POST /api/auth/register.
3. Spring Security hashes the password using BCrypt.
4. User entity is saved to the users table in PostgreSQL.
5. JWT access token is generated and returned to the client.
6. Subsequent requests include the token in the Authorization header.

Product Browsing & Search

7. User visits the homepage; React fetches GET /api/products.
8. Spring Boot queries the products table with optional filters (category, brand, price range).
9. Paginated product list is returned as JSON and rendered by React.
10. User can filter by category, brand, price, or rating in real time.

Cart & Checkout

11. User clicks 'Add to Cart'; product is stored in React local state / Context.
12. Cart summary shows itemized list with auto-calculated subtotal and GST.
13. User clicks 'Checkout'; React calls POST /api/orders to initiate order.
14. Spring Boot creates an Order record with status PENDING.
15. Razorpay order is created via the Java SDK; payment UI is presented.
16. On payment success, Razorpay sends a webhook to POST /api/payment/verify.
17. Spring Boot verifies the signature, updates order status to PAID and PROCESSING.
18. Stock is decremented in the products table.
19. Order confirmation is returned to the React frontend.





5. Key REST API Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user	Public
POST	/api/auth/login	Login and receive JWT	Public
GET	/api/products	Get all products (with filters)	Public
GET	/api/products/{id}	Get product details	Public
POST	/api/products	Add new product	Admin
PUT	/api/products/{id}	Update product	Admin
DELETE	/api/products/{id}	Delete product	Admin
POST	/api/orders	Place new order	User
GET	/api/orders/my	Get current user's orders	User
GET	/api/admin/orders	Get all orders	Admin

PATCH	/api/admin/orders/{id}	Update order status	Admin
POST	/api/payment/verify	Verify Razorpay payment webhook	Public
POST	/api/reviews	Submit a product review	User

6. Security and Performance Considerations

6.1 Security

- All passwords stored as BCrypt hashes — raw passwords never persisted.
- JWT tokens include expiry (15 min access / 7 day refresh) to limit exposure.
- Spring Security enforces role-based access: USER vs ADMIN routes are strictly separated.
- Razorpay webhook signature is verified using HMAC-SHA256 before updating order status.
- Input validation via Spring's @Valid and Bean Validation annotations prevents injection attacks.
- HTTPS enforced on all production endpoints via Railway and Vercel configurations.

6.2 Performance

- HikariCP connection pooling minimizes database connection overhead.
- Pagination implemented on all list endpoints to avoid large data payloads.
- React lazy loading and code splitting for faster frontend load times.
- Database indexes on frequently queried columns: email, category_id, product name.

7. Deployment Architecture

Component	Platform	Details
React Frontend	Vercel	Auto-deploys from GitHub main branch; global CDN
Spring Boot Backend	Railway	Dockerized Spring Boot app; environment variables for secrets
PostgreSQL Database	Supabase	Managed cloud PostgreSQL; connection via JDBC URL
Static Assets	Cloudinary (optional)	Product image storage and CDN delivery
Payment Gateway	Razorpay Sandbox	Test mode for development; live keys for production

The React frontend is served from Vercel's global CDN. It communicates with the Spring Boot REST API hosted on Railway via HTTPS. The API connects to Supabase PostgreSQL using a secure JDBC connection string stored as an environment variable.

8. Conclusion

The TechNest system architecture is designed as a scalable, secure, and maintainable three-tier application. The choice of Java + Spring Boot for the backend brings enterprise-grade reliability, robust security via Spring Security, and clean data management through Hibernate JPA. PostgreSQL's relational model ensures data integrity across users, products, orders, and reviews. The defined process flows, REST API structure, and deployment strategy provide a clear and complete roadmap for the development phases that follow in subsequent experiments.